Домашнее задание ко второй лекции курса "Тензорные компиляторы"

Шелонин Арсений Б01-411

Вариант 1

```
int function1(int* a, int n) {
    int result = 0;
    for (int i = 0; i < n; ++i) {
        int x = 5;
        for (int j = 0; j < n; ++j) {
            if (j % 2 == 0)
                continue;
            result += 2 * 3 * a[j] * x - i - 2;
        }
    }

    return result;
}
```

Что делает: Аргументами принимает указатель на int a и целое n, возвращает $int\ result$. Эта программа проходит внешним циклом от 0 до n и внутренним от 0 до n по нечетным значениям счетчика j. К переменной $result$ прибавляется $2 * 3 * a[j] * x - i - 2$.

1 SSA IR

Теперь нам для построения Generic IR и потока управления (пока что не в SSA форме) нужно перевести цикл программы в do-while представление:

1.1 Do-While представление

```
int function1(int* a, int n) {
    int result = 0;

    int i = 0;
    if (i >= n) {
        return result;
    }

    do {
        int x = 5;

        int j = 0;
        if (j < n) {
            do {
                if (j % 2 == 0) {
                    j++;
                    continue;
                }

                result += 2 * 3 * a[j] * x - i - 2;
                j++;
            } while (j < n);
        }

        i++;
    } while (i < n);

    return result;
}
```

1.2 Generic IR без SSA формы

```
int function1(int* a, int n) {
  func_start:
    result = 0;
    i = 0;

    loop1_neg_cnd = (i >= n);
    br loop1_neg_cnd, exit, loop1_body;

  loop1_body:
    x = 5;
    j = 0;

    loop2_cnd = (j < n);
    br loop2_cnd, loop2_head, loop1_end;

  loop2_head:
    e = (j % 2);
    f = (e == 0);
    br f, loop2_end, loop2_body;

  loop2_body:
    tmp = 2 * 3;

    addr = a + j;
    val = *addr;

    tmp = tmp * val;
    tmp = tmp * x;
    tmp = tmp - i;
    tmp = tmp - 2;
    result = result + tmp;
    br loop2_end;

  loop2_end:
    j = j + 1;

    loop2_cnd = (j < n);
    br loop2_cnd, loop2_head, loop1_end;

  loop1_end:
```

```
    i = i + 1;
    loop1_cnd = (i < n);
    br loop1_cnd, loop1_body, exit;

  exit:
    return result;
}
```

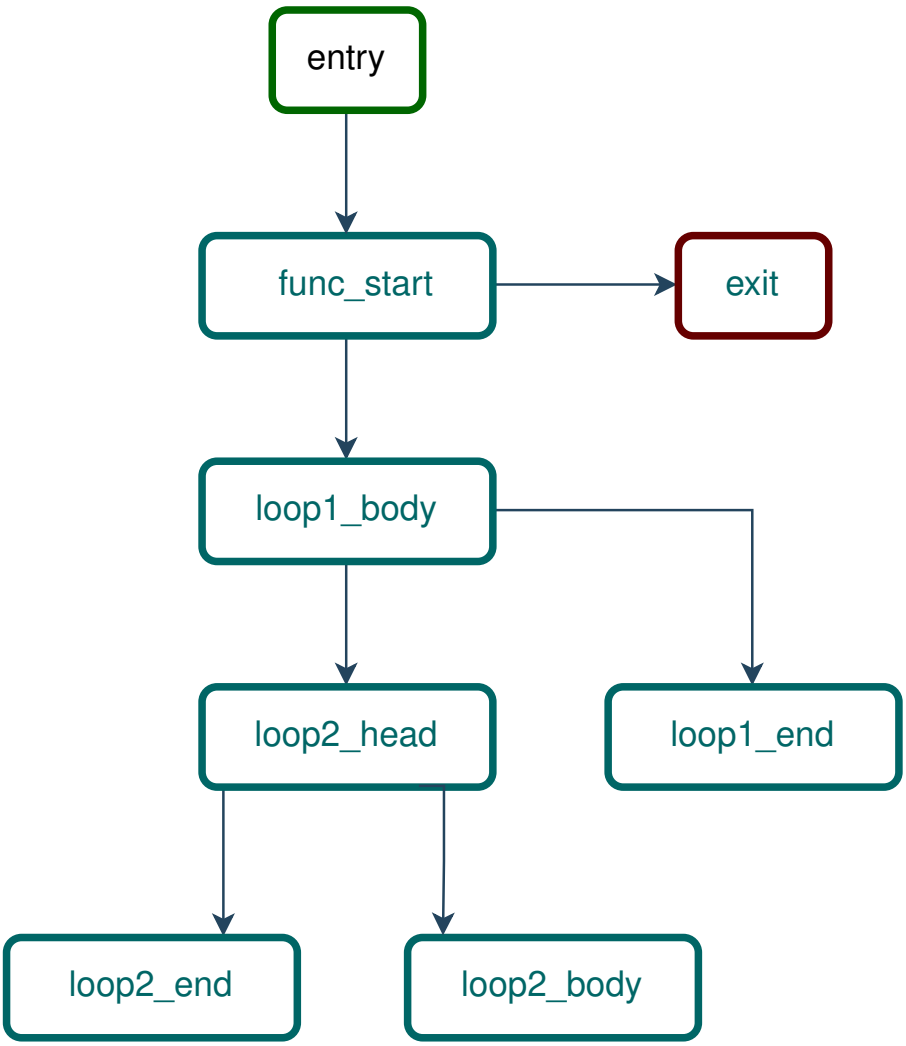
entry
func_start
exit
loop1_body
loop2_head
loop1_end
loop2_end
loop2_body

Рис. 2: Dom(CFG)

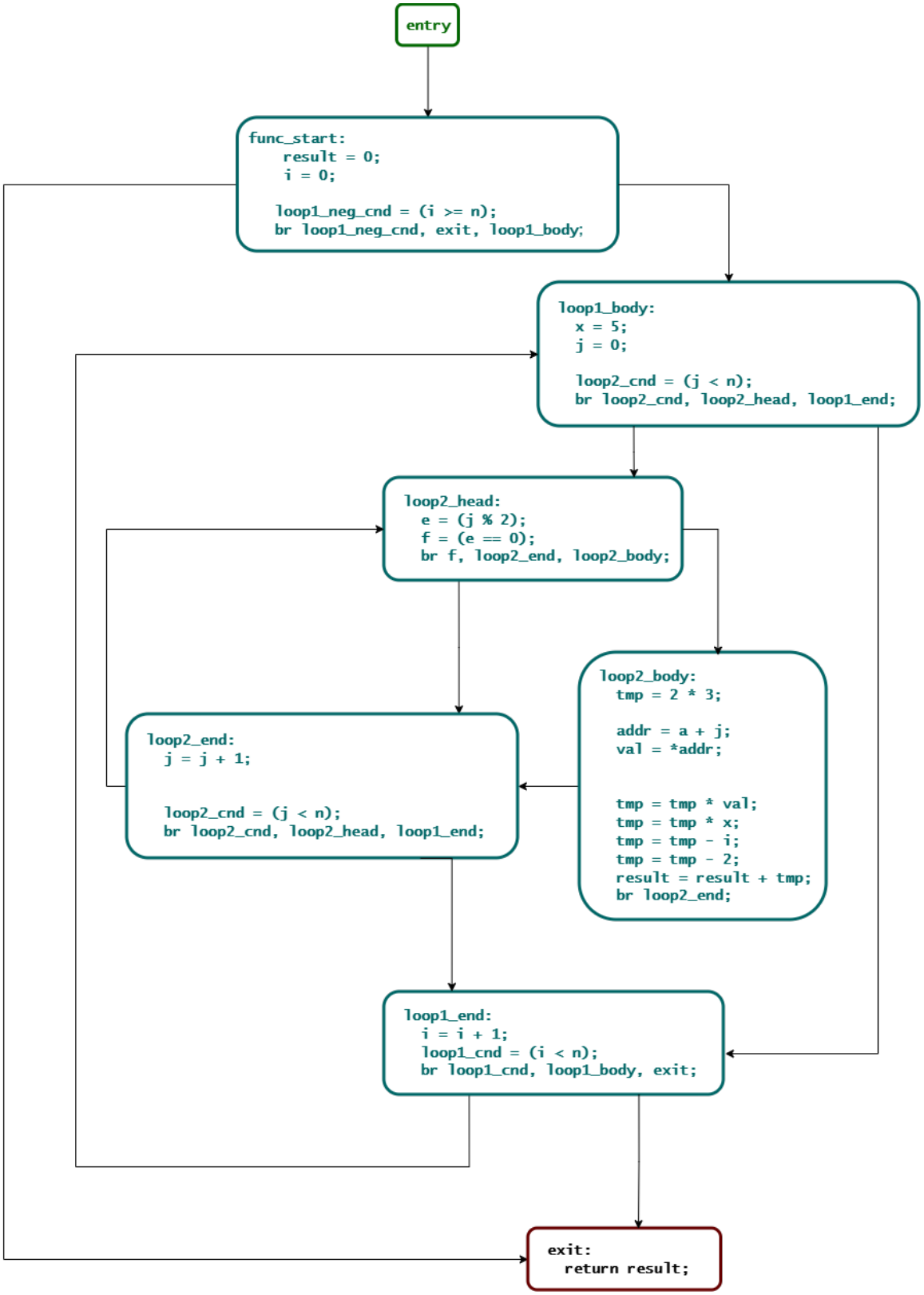

Рис. 1: CFG без SSA формы

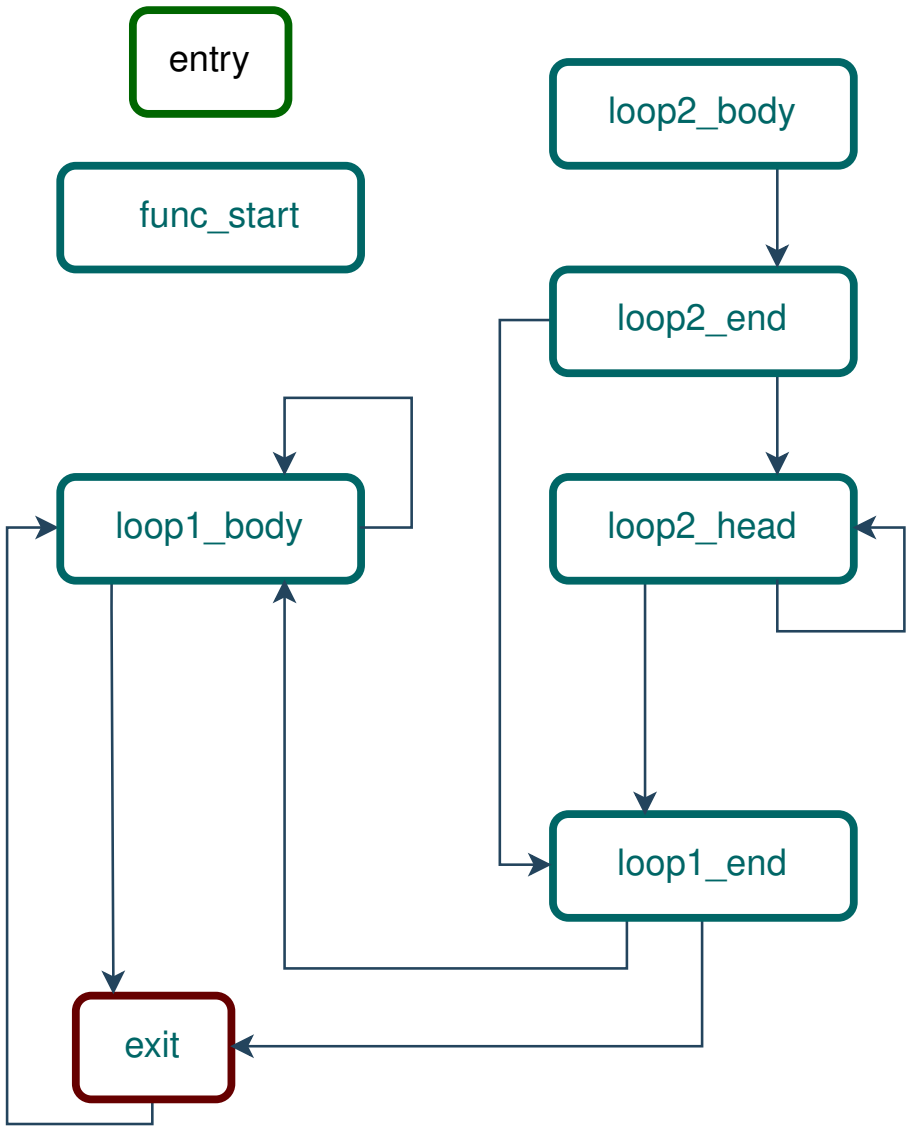
entry
func_start
loop2_body
loop2_end
loop1_body
loop2_head
loop1_end
exit

Рис. 3: DF(CFG)

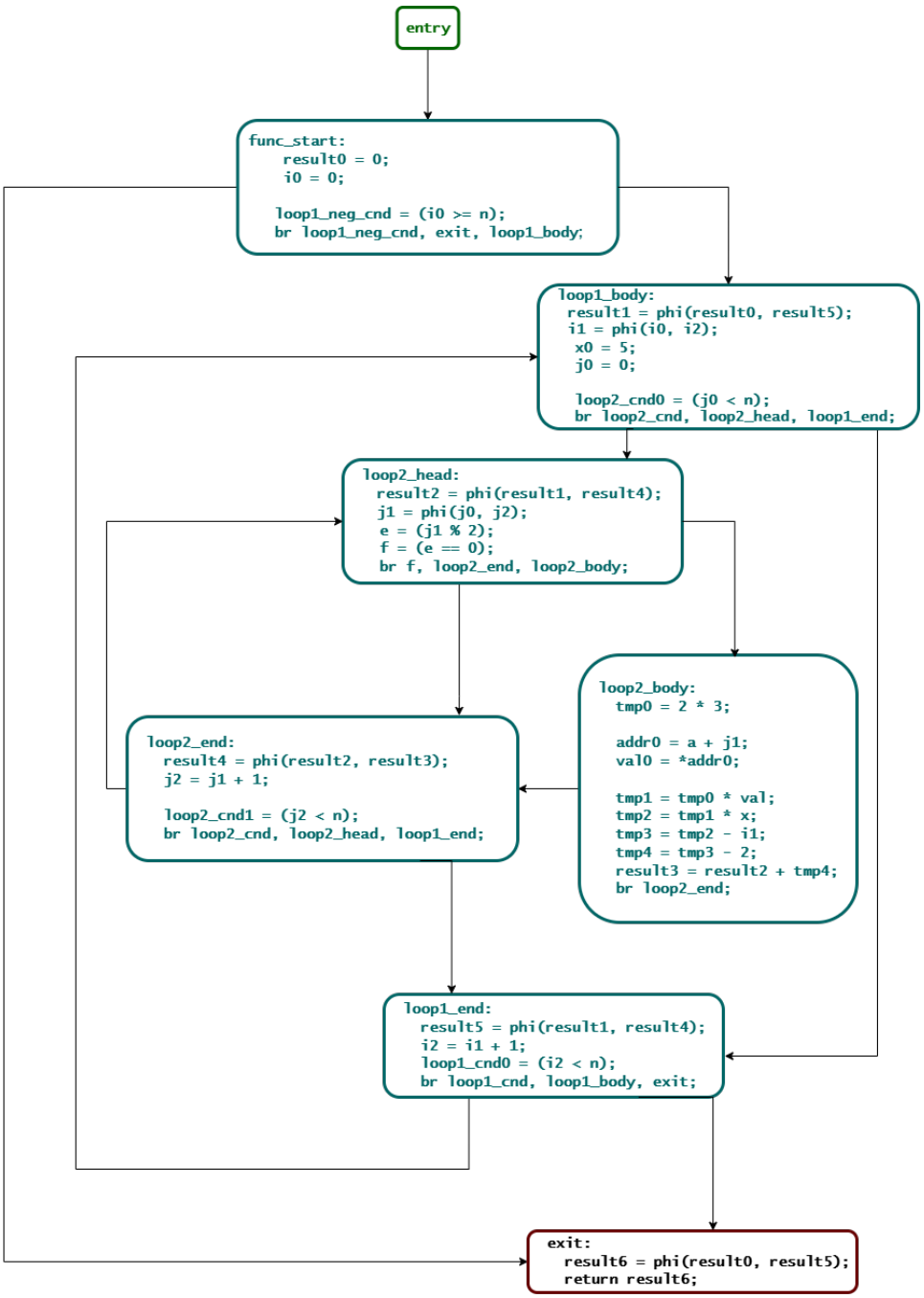

Рис. 4: CFG with SSA

1.3 IR в SSA форме

```
int function1(int* a, int n) {
  func_start:
    result0 = 0;
    i0 = 0;
    loop1_neg_cnd0 = (i0 >= n);
    br loop1_neg_cnd0, exit, loop1_body;

  loop1_body:
    result1 = phi(result0, result5);
    i1 = phi(i0, i2);
    x0 = 5;
    j0 = 0;
    loop2_cnd0 = (j0 < n);
    br loop2_cnd0, loop2_head, loop1_end;

  loop2_head:
    result2 = phi(result1, result4);
    j1 = phi(j0, j2);
    e0 = (j1 % 2);
    f0 = (e0 == 0);
    br f0, loop2_end, loop2_body;

  loop2_body:
    tmp0 = 2 * 3;
    addr0 = a + j1;
    val0 = *addr0;
    tmp1 = tmp0 * val0;
    tmp2 = tmp1 * x0;
    tmp3 = tmp2 - i1;
    tmp4 = tmp3 - 2;
    result3 = result2 + tmp4;
    br loop2_end;

  loop2_end:
    result4 = phi(result2, result3);
    j2 = j1 + 1;
    loop2_cnd1 = (j2 < n);
    br loop2_cnd1, loop2_head, loop1_end;

  loop1_end:
```

```
    result5 = phi(result1, result4);
    i2 = i1 + 1;
    loop1_cnd0 = (i2 < n);
    br loop1_cnd0, loop1_body, exit;

  exit:
    result6 = phi(result0, result5);
    return result6;
}
```

2 Оптимизации

2.1 Constant propagation + Алгебраическое переупорядочивание + Constant folding

В блоке $loop2_body$ можем подставить известное на этапе компиляции константное значение $x = 5$ (соответственно убрав определение $x0$ в $loop1_body$), затем переупорядочить операции умножения так, что ранг констант минимален, и свернуть выражения умножения с константными аргументами:

до оптимизации

```
loop2_body:
    tmp0 = 2 * 3;
    addr0 = a + j1;
    val0 = *addr0;
    tmp1 = tmp0 * val0;
    tmp2 = tmp1 * x0;
    tmp3 = tmp2 - i1;
    tmp4 = tmp3 - 2;
    result3 = result2 + tmp4;
    br loop2_end;
```

после

```
loop2_body:

    addr0 = a + j1;
    val0 = *addr0;
    tmp0 = 30 * val0;
    tmp1 = tmp0 - i1;
    tmp2 = tmp1 - 2;

    result3 = result2 + tmp2;
    br loop2_end;
```

Полный код

```
int function1(int* a, int n) {
  func_start:
    result0 = 0;
    i0 = 0;
```

```
    loop1_neg_cnd0 = (i0 >= n);
    br loop1_neg_cnd0, exit, loop1_body;

  loop1_body:
    result1 = phi(result0, result5);
    i1 = phi(i0, i2);
    j0 = 0;
    loop2_cnd0 = (j0 < n);
    br loop2_cnd0, loop2_head, loop1_end;

  loop2_head:
    result2 = phi(result1, result4);
    j1 = phi(j0, j2);
    e0 = (j1 % 2);
    f0 = (e0 == 0);
    br f0, loop2_end, loop2_body;

  loop2_body:

    addr0 = a + j1;
    val0 = *addr0;
    tmp0 = 30 * val0;
    tmp1 = tmp0 - i1;
    tmp2 = tmp1 - 2;

    result3 = result2 + tmp2;
    br loop2_end;

  loop2_end:
    result4 = phi(result2, result3);
    j2 = j1 + 1;
    loop2_cnd1 = (j2 < n);
    br loop2_cnd1, loop2_head, loop1_end;

  loop1_end:
    result5 = phi(result1, result4);
    i2 = i1 + 1;
    loop1_cnd0 = (i2 < n);
    br loop1_cnd0, loop1_body, exit;

  exit:
```

```
    result6 = phi(result0, result5);
    return result6;
}
```

3 Цикловые оптимизации

Для начала, после лекции по цикловым оптимизациям, дадим базовым блокам более осмысленные имена.

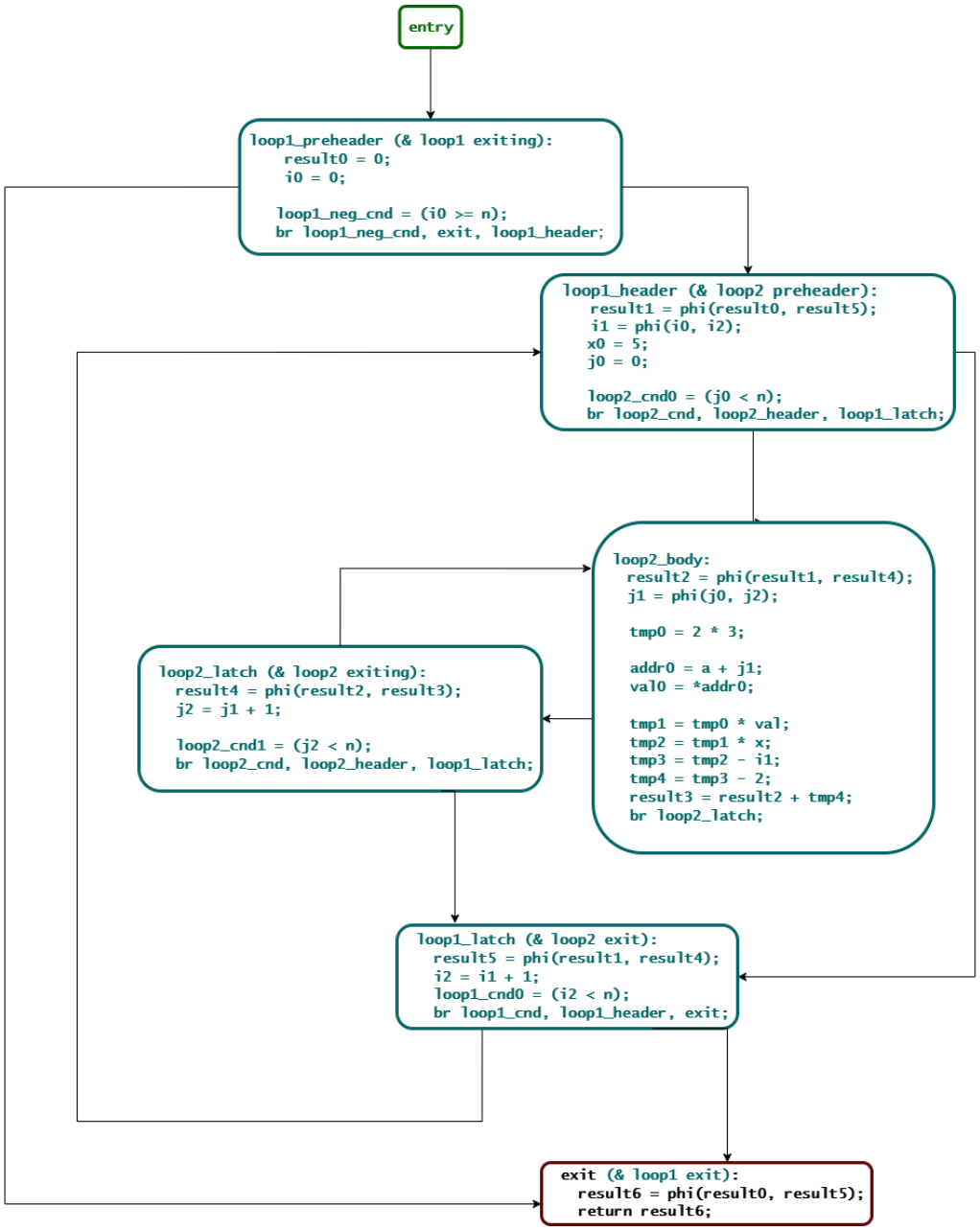

Рис. 5: CFG with SSA

3.1 Loop Strength Resuction

Заметим, что в внутреннем цикле индуктивная переменная $\mathtt{j} =< 0, +, 1 >$, при этом внутри стоит проверка на $\mathtt{j}\%2 == 0$. В таком случае можем заменить индуктивность на $\mathtt{j} =< 1, +, 2 >$, а лишнюю проверку выкинуть.

CFG цикл примет вид

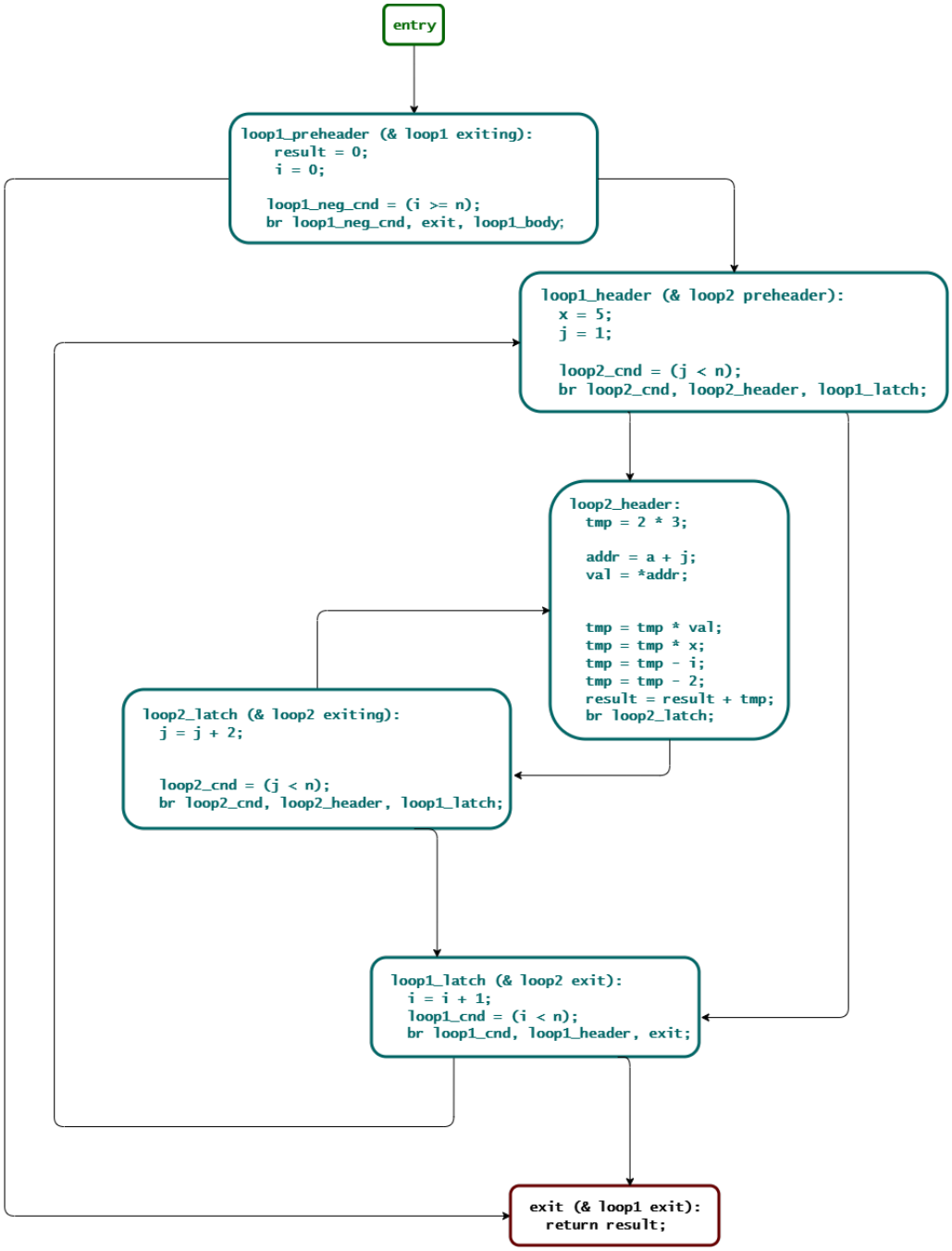

Рис. 6: CFG with SSA